

Множественная навигация в списочных контролах

Множественная навигация используется в иерархических списках для загрузки с уже развёрнутыми узлами или для перезагрузки с сохранением состояния развёрнутости узлов.

Взаимодействие списочных контролов с источником данных ведётся через специальный контроллер, который обеспечивает работу навигации, формирует параметры для запроса и т.д. Он хранит навигацию и для узлов, а также правильно формирует параметры навигации для запроса.

Про настройку контрола на мультинавигацию можно прочитать [здесь](#).

При выполнении запроса с множественной навигацией в параметрах списочного метода в навигации передаётся RecordSet с форматом:

- **id** — идентификатор узла.
- **nav** — навигация для этого узла.

```
1 // Пример запроса
2 params: {
3     Филтр: {...},
4     Сортировка: null,
5     Навигация: [
6         0: {
7             id: null,
8             nav: {
9                 ЕстьЕще: true,
10                РазмерСтраницы: 45,
11                Страница: 0
12            }
13        },
14        1: {
15            id: 1371,
16            nav: {
17                ЕстьЕще: true,
18                РазмерСтраницы: 45,
19                Страница: 0
20            }
21        },
22        2: {
23            id: 14463814,
24            nav: {
25                ЕстьЕще: true,
26                РазмерСтраницы: 45,
27                Страница: 0
28            }
29        }
30    ]
31    ДопПоля: [ ]
32 }

1 // Пример ответа бэка на первый запрос
2 {
3     "jsonrpc": "2.0",
4     "result": {
```

```

5      "f": 0,
6      "d": [...],
7      "s": [...],
8      "_type": "recordset",
9      "n": {
10         "f": 0,
11         "d": [
12             [null, false],
13             [1371, true], // Для этого раздела записи есть еще
14             [14463814, false]
15         ],
16         "s": [{
17             "n": "id",
18             "t": "Число целое"
19         }, {
20             "n": "nav_result",
21             "t": "JSON-объект"
22         }],
23         "_type": "recordset"
24     },
25     "id": 1,
26     "protocol": 6
27 }

```

Для узла сохраняется не только навигация по факту его раскрытия, но и остальные аспекты, например, для постраничной навигации будет сохранено количество страниц, которые были загружены при клике по [кнопке "Еще"](#) в узле.

В ответе метод возвращает информацию о навигации для каждого узла в виде RecordSet'a с форматом:

- **id** — идентификатор узла
- **nav_result** — результат навигации

Данные всех развернутых узлов необходимо вернуть с БЛ в виде RecordSet. В мета-информации RecordSet должна содержаться информация для навигации по развернутым узлам. Например, если в фильтре запроса присутствуют идентификаторы раскрытых узлов ([expandedItems](#)) disk и recent:

```

1  const filter = {
2    ...,
3    folder: [
4      'disk',
5      'recent'
6    ]
7    ...,
8  }

```

то ответ БЛ должен содержать следующую информацию о мультинавигации:

```

1  more: [
2    ...,
3    {
4      id: "disk",
5      nav_result: {
6        after: false,
7        before: false
8      }
9    },
10   {
11     id: "recent",

```

```

12         nav_result: {
13             after: false,
14             before: false
15         }
16     }
17     ...,
18 ]

```

Пример реализации списочного метода бизнес-логики на Python

```

1  # Python
2  MULTI_CURSOR_FRMT = CreateRecordFormat()
3  MULTI_CURSOR_FRMT.AddString('id')
4  MULTI_CURSOR_FRMT.AddJson('nav_result')
5  def Test.MultiRoot(ДопПоля, Фильтр, Сортировка, Навигация):
6      rs = RecordSet(CurrentMethodResultFormat())
7      if Навигация.Type() == NavigationType.ntMULTI_ROOT:
8          nav_result = {}
9          multi_cursor = CreateRecordSet(MULTI_CURSOR_FRMT)
10         for id, nav in Навигация.RootsDict().items():
11             # Запрашиваем данные по одному разделу.
12             Фильтр.Раздел = id
13             tmp_rs = Test.MultiRoot(ДопПоля, Фильтр, Сортировка, nav)
14             # Склеиваем результаты.
15             for rec in tmp_rs:
16                 rs.AddRow(rec)
17             # Формируем общий результат навигации по всем разделам.
18             nav_result[ id ] = tmp_rs.nav_result
19             cursor = multi_cursor.AddRow()
20             cursor['id'].From(id)
21
22         cursor['nav_result'].From(tmp_rs.Metadata().Get("nextPosition"))
23         rs.nav_result = NavigationResult(nav_result)
24         rs.Metadata().AddRecordSet("nextPosition", MULTI_CURSOR_FRMT,
25         multi_cursor)
26     else:
27         # Тут обработка обычной навигации, например, вызов
28         декларативного списка.
29         rs = Test.DeclList(ДопПоля, Фильтр, Сортировка, Навигация)
30     return rs

```